

AI Trust: Practice Theory — The Implementation

Monyet Batu

ORCID: 0009-0007-9002-5381

27 May 2026

A follow-up to [AI Trust and Situated Awareness: A Practice Theory Reframe](#) and [Practice Theory — The Apprenticeship and a Strange Loop](#).

Reader's note: this is the long technical paper of the series. The two prior essays make the conceptual case for the architecture in short prose; readers who want only the high-level argument can stop there. This essay shows the build itself, twelve steps long, alongside an accompanying repository.

*A note on the steps: each step describes what it contributes to the architecture, and identifies the files that work touches. The repository reflects the **final state** at the end of step 12 — every file named in an early step exists at HEAD in its fully-evolved form, with everything later steps added to it. A single "Run with" instruction appears once, at the end of step 11, exercising the complete verify. The journey is in the prose; the artifact is at HEAD.*

From theory to substrate

The previous essays describe how AI Trust can be engineered by applying practice theory. The **situated awareness** an LLM needs to act as a practitioner — its grasp of where it is in the work, what the goal is, what is at hand, what would be a legitimate next step — is captured in a **practice recipe** that apprentices the harness LLM in both the practice itself and the user it serves; the LLM, in enacting the recipe, becomes the practitioner. The arrangement is then made self-maintaining and self-improving through an autonomic loop that observes the system's own operation.

This essay turns those ideas into a working implementation, built step by step alongside an accompanying repository.

A note on terminology

In the previous essays I used **practice recipe** to describe the five-element structure that lets an LLM be cued into a practice — what was earlier framed as a "meaning layer" and is more precisely the captured form of a practitioner's **situated awareness**. Having sat with the ideas for a while, I now describe these as a **practice bundle** in Schatzki's inner sense — the interlocked ensemble of the five constitutive elements — not in the practice-cluster sense the term often carries.

The change is more than cosmetic. *Recipe* was carrying two things at once: the thing transmitted, and the means of transmission. *Bundle* names only the thing transmitted; **apprenticeship** names the means. A human apprentice receives a practice bundle through years of bodily participation; an LLM apprentice receives a practice bundle through engaging with its captured form. Same process, two media. So:

An intelligence receives a practice bundle through apprenticeship and from that is able to enact the practice; in doing so the intelligence becomes the practitioner.

The rest of this essay uses *bundle* throughout.

The plan

The build is taken step by step. Each step adds one piece to the working system and is described as that piece arrives, so the essay and the implementation grow in step. The plan, at a high level, is:

- 1. **Capture a single practice as a bundle.** A self-contained inscription of one practice with its five elements.
- 2. **Pools and the function registry.** Sharing elements across bundles; binding a material's named function to executable code.
- 3. **Projection.** Assembling a runnable practice from a bundle and the pools at request time.
- 4. **The practice server.** Presenting the projected practice to a harness through MCP — a fixed tool surface whose affordances change with the active practice. The standing arrangement that turns this into the *apprenticeship server* of the prior essays arrives in step 6.
- 5. **Engagement and enactment.** Sessions, the inspectable trail, the recorded steps that make trust enacted structure rather than only messaging.
- 6. **The apprenticeship layer.** The standing arrangement within which practices are taken up and set down; the engagement bundle that wraps them.
- 7. **Practice Management.** The mutable substrate, the meta-materials, and the somatic practice for authoring and amending the substrate at runtime — the "recipe for making recipes" doc 2 named as the meta-practice.
- 8. **Somatic and autonomic modes.** Splitting the surface so practices that need the user are kept distinct from practices that can act alone.
- 9. **The Judge.** Primitives for reading the trail and emitting Friction observations; the heuristics live in the bundle's understanding, enacted by an LLM.
- 10. **The Smoother.** Primitives for reading Friction and marking it addressed; the amendment work reuses Practice Management's affordances. Interpretation lives in the bundle's understanding, enacted by an LLM.
- 11. **The autonomic harness.** Inbox + dispatcher pattern; an adapter abstraction over the LLM-driving primitive with concrete subclasses across two providers (Anthropic, Codex) and two process shapes (long-lived in-process, subprocess per dispatch); one run-loop, three real adapters.
- 12. **The strange loop.** Judge and Smoother are practices too; they can be read and amended in turn — through the very meta-materials they were built with.

Read in order. Each step assumes the steps before it and prepares the steps after.

Step 1: Capturing a single practice

The captured shape

A single practice, captured as a bundle, has this shape:

```
Bundle
  id:                uniquely identifies the practice bundle
  name:              names the practice defined by the practice bundle
  description:       describes the practice defined by the practice bundle
  ---
  teleo_affective :  prose describing the ends and orientation
  understanding   :  prose describing what a practitioner knows
```

```

rules      : a list of explicit rules
affordances : a pool of Affordance
materials  : a pool of Material

Affordance
  id      : uniquely identifies the affordance
  name    : names the affordance
  description : what becomes possible, framed from the practice's
perspective
  ---
  materials : one or more Material this affordance reaches for,
referenced by name

Material
  name      : unique name; affordances reference it by this, and
step 2's registry binds it by this
  description : describes the material
  input_schema : the arguments the function accepts, as a schema

```

Pooling affordances and materials

Within a bundle, affordances and materials are both pooled — a pool of affordances at the bundle level, a pool of materials at the bundle level. Affordances reference materials by name rather than nesting them.

There are two reasons for this.

The **technical reason** is to avoid duplication. If two affordances reach for the same material, we do not want to write that material twice. The material is defined once in the bundle's material pool, and affordances reference it by name.

The **practice-theoretic reason** is that the same material is often afforded differently. A chisel in a workshop is the same physical tool whether it is being used to cut a joint, clean out a mortice, or trim a dowel — but the affordance the practitioner reaches for is different in each case. The material does not change; the framing does. Pooling materials at the bundle level lets a single material be afforded multiple ways by different affordances in the same bundle, which matches how practices actually work.

In the worked example below, `garmin_list_activities` sits in the bundle's material pool once and is reached for by two affordances — `recent_activity` and `intermittent_walking_analysis` — each affording it differently. Step 2 extends this pooling pattern further, sharing pools across bundles. Within a single bundle, the pooling is already in place.

A worked example: Activities Management

To make the shape concrete, here is a captured practice in full. *Activities Management* helps a user see their physical activities clearly — the record of what they did and the rhythm those activities are forming. The materials reach a wearable device's API, which in this case is Garmin.

```

Bundle
  id      : activities_management
  name    : Activities Management

```

description : Keep an honest, useful view of the user's physical activities –

what's been done, what the body is showing, what the rhythm looks like.

teleo_affective:

Help the user see their activities clearly – the record of what they did

and the rhythm those activities are forming. Stay an observer and recorder,

not a coach. The user's body and choices are sovereign; the practice serves

their seeing, not their compliance.

understanding:

An activity has a type, a duration, sometimes a distance, an intensity, a

heart-rate band, and a perceived effort. Activities accumulate into patterns: training load, weekly volume, recovery debt. Garmin is the source

of truth for tracked activities; gaps are filled by what the user tells us.

The user's questions vary in shape – "what did I do yesterday", "how is

this week going", "am I overdoing it" – and each calls for a different read

of the same underlying record.

rules:

- Cite the source of any datum (device-tracked, manual, derived).
- Do not infer training intent from raw data – ask if framing matters.
- Do not give prescriptive coaching unless asked.
- Never expose activity data outside the apprenticeship.

affordances:

– id : recent_activity
 name : Recent activity
 description : Look at activities over a recent window (today, last 7 days, etc.)
 materials : [garmin_list_activities]

– id : activity_detail
 name : Activity detail
 description : Review one activity in detail – splits, heart rate, route.
 materials : [garmin_get_activity]

– id : daily_summary
 name : Daily summary
 description : See the day's overall picture – steps, sleep, stress, body battery.
 materials : [garmin_get_daily_summary]

– id : intermittent_walking_analysis

```

    name      : Intermittent walking analysis
    description : Analyse the user's IWT sessions – fast/slow interval
recognition,
                time-in-fast vs time-in-slow, weekly fast minutes,
progression
                over recent weeks.
    materials  : [ garmin_list_activities, garmin_get_activity,
garmin_get_user_stats ]

materials:
  - name      : garmin_list_activities
    description : List the user's activities within a date range.
    input_schema : { start_date: date, end_date: date, activity_type?:
string }

  - name      : garmin_get_activity
    description : Fetch full detail for a single activity by its
Garmin ID.
    input_schema : { activity_id: string }

  - name      : garmin_get_daily_summary
    description : Fetch the daily wellness summary for a given date.
    input_schema : { date: date }

  - name      : garmin_get_user_stats
    description : Fetch aggregate stats (volume, distance, time-in-
zones) for a period.
    input_schema : { start_date: date, end_date: date }

```

In the implementation accompanying this essay these four materials are mocked — each returns synthetic data parameterised by date — so the focus stays on the bundle shape rather than the Garmin integration.

A few things to notice in the captured form.

The **teleo-affective** does not describe the API or the data. It describes a stance: observer, not coach; the user is sovereign; the practice serves their seeing. An LLM engaging this bundle is being cued into that posture before any function is mentioned.

The **understanding** carries the practical knowledge that lets the practitioner read a question and know what shape of answer it calls for. The shape-of-question observation — *what did I do yesterday* vs *how is this week going* vs *am I overdoing it* — is not a rule and not an affordance. It is the kind of thing an apprentice would absorb through participation. Here, it has to be written.

The **rules** are short and independent. Each one names a discipline the practitioner is to hold while enacting any affordance. They are not workflow steps and they are not preferences. They are the bright lines.

The **affordances** are high-level. *Intermittent walking analysis* is a thing a user might reasonably want to look at; it is not a function call. The same material — `garmin_list_activities` — appears under two affordances (`recent_activity` and `intermittent_walking_analysis`) because both reach for the same mechanical capability, framed differently. That re-use is the point of separating affordances from materials.

The **materials** describe a small, plain surface — four functions, each with a description and an argument shape. The bundle says what these functions do and what they take; it does not say how they are implemented. The implementation lives behind a name.

What step 1 contributes

Step 1 establishes:

- The **captured bundle shape** in `src/practice_theory_implementation/types.py` (`Bundle`, `Affordance`, and `Material` dataclasses).
- The **first worked bundle** — Activities Management — in `src/practice_theory_implementation/bundles/activities_management.py`, as Python literals matching the worked example field-for-field.
- The **mock materials** the bundle reaches for in `src/practice_theory_implementation/materials/garmin_mock.py`, four functions parameterised by date for stable per-day output.

These files grow as later steps extend them: step 2 refactors `Bundle` to be a selection over pools, step 8 adds a `mode` field, and `__main__.py` (first written here to exercise the mocks) is rewritten progressively until it drives the full verify at step 11.

What step 1 leaves for later

A single captured bundle is the smallest unit. On its own it does not yet say how elements get shared across bundles, how a material's name gets bound to runnable code, how the captured form gets projected into something an LLM can engage with, or how an enactment leaves a trail behind it. Each of those is its own step, and each gets its own section as we build it.

The bundle is what the apprentice would carry into the workshop. The workshop, the bench, the tool cupboard, and the standing arrangement around them are all coming — but the bundle comes first.

Step 2: Pools and the function registry

Step 1 left two threads hanging. First, every bundle holds its own teleo-affective, understanding, rules, and affordances inline — fine for one bundle, awkward when the same content needs to appear in another. Second, materials reference functions by name, but nothing yet resolves those names into callable code. Step 2 picks both threads up: pools that lift the constitutive elements out of any one bundle, and a function registry that binds material names to executable code.

Pools across bundles

Within a single bundle, affordances and materials are already pooled (step 1). Step 2 extends the pattern outward. The five constitutive elements — teleo-affective, understanding, rules, affordances, materials — live at the substrate level, shared across all bundles. A bundle stops carrying its own content inline and becomes a selection over the pools.

```
Substrate
teleo_affective : pool of PoolElement
understanding   : pool of PoolElement
```

```

rules      : pool of PoolElement
affordances : pool of Affordance
materials  : pool of Material

```

The teleo-affective, understanding, and rules pools all hold the same shape — small content elements with an id, a name, and a body of prose. They share a single type:

```

PoolElement
  id      : unique handle within its pool
  name    : human-readable name
  content : the prose this element carries

```

Affordances and materials keep the types they had in step 1 (each carries more structure than a PoolElement does), but they now live at the substrate level rather than inside a bundle.

Bundle as a selection

With the pools in place, a bundle is no longer a body of content. It is a selection of IDs into the pools:

```

Bundle
  id      : uniquely identifies the bundle
  name    : names the practice
  description : describes the practice
  ---
  teleo_affective_ids : IDs into the teleo-affective pool
  understanding_ids   : IDs into the understanding pool
  rules_ids           : IDs into the rules pool
  affordance_ids      : IDs into the affordance pool

```

Materials do not have their own ID list on the bundle. The bundle's effective materials are derived from its affordances — every material an affordance reaches for is part of the bundle, and a material no affordance reaches for is not. This matches the practice-theoretic shape: an affordance frames the practice's vocabulary, and the materials follow from what the affordances reach for.

Because the pools are at the substrate level, sharing happens naturally across bundles as well as within. The same material in the materials pool can be reached for by affordances in any bundle; the same affordance in the affordance pool can be referenced by any bundle that wants it. A material or affordance defined once is available to whichever bundles select it. This is the cross-bundle reuse that step 1's within-bundle pooling pointed at.

The function registry

The function registry is the binding from a material's name to the executable that runs when the material is invoked:


```
Registry
dict[str, Callable]    # keyed by Material.name
```

The registry is populated by hand at module load time. A `registry.py` module imports the mock functions and binds each one explicitly. No decorator, no auto-discovery — the binding is visible in one place. Step 2's job is to make it work, not to make it ergonomic.

The separation of registry from material is what step 1 promised. The bundle describes; the registry executes. A material's captured surface (description, input schema) can be amended without touching the executable it binds to; the executable can be swapped (mock for real) without touching the captured surface. Step 2 makes that separation concrete.

Dynamic creation of materials without restart is still supported. The registry is an ordinary mutable dict; new bindings can be installed at runtime, and a runtime-authored material file can register its captured surface into the materials pool and its callable into the registry in one import. A decorator would collapse those two registrations into a single declaration if and when authoring ergonomics warrant it. For step 2, the hand-built form keeps the binding plain.

Re-expressing Activities Management

The captured Activities Management bundle from step 1 stays exactly the same as a description of the practice. It changes only in how it is *expressed*. Each element moves into its pool with an id; the bundle becomes a selection of those ids.

The bundle's teleo-affective prose becomes a single PoolElement:

```
PoolElement
  id      : te_activities_management
  name    : Observer of activities
  content : Help the user see their activities clearly – the record of
what
           they did and the rhythm those activities are forming. Stay an
           observer and recorder, not a coach... (full prose as in step
1)
```

Each of the four rules from step 1 becomes its own PoolElement so the rules pool can be examined entry by entry (a Judge in step 9 will read them against the enactment trail independently):

```
PoolElement
  id      : rule_cite_source
  name    : Cite source of any datum
  content : Cite the source of any datum (device-tracked, manual,
derived).
```

The four affordances move into the affordances pool with the same shape they had in step 1. The four materials move into the materials pool. And the bundle becomes a thin selection:


```

Bundle
  id          : activities_management
  name        : Activities Management
  description  : Keep an honest, useful view of the user's physical
activities -
                what's been done, what the body is showing, what the
rhythm looks like.
---
teleo_affective_ids : ( te_activities_management, )
understanding_ids   : ( und_activities_management, )
rules_ids           : ( rule_cite_source, rule_no_intent_inference,
                        rule_no_coaching, rule_no_external_exposure )
affordance_ids      : ( recent_activity, activity_detail, daily_summary,
                        intermittent_walking_analysis )

```

Behind that selection, the registry binds each material name to its mock function:

```

Registry
garmin_list_activities  -> garmin_mock.garmin_list_activities
garmin_get_activity    -> garmin_mock.garmin_get_activity
garmin_get_daily_summary -> garmin_mock.garmin_get_daily_summary
garmin_get_user_stats  -> garmin_mock.garmin_get_user_stats

```

When an affordance is invoked, the resolution path is: bundle → affordance → material name → registry → callable. Step 3 (projection) will tighten this into a single invocation surface.

What step 2 contributes

Step 2 establishes:

- The **substrate** type (`PoolElement + Substrate`) and the refactored **Bundle as a selection of IDs** in `types.py`; the old `material_by_name / validate()` methods come off **Bundle** and become a free `validate_bundle(bundle, substrate)` function.
- The **seed pools** in `src/practice_theory_implementation/pools.py`, hand-populating `TELEO_AFFECTIVE`, `UNDERSTANDING`, `RULES`, `AFFORDANCES`, `MATERIALS` with the content Activities Management selects from.
- The **function registry** in `src/practice_theory_implementation/registry.py`: a module-level `FUNCTIONS: dict[str, Callable]` with `register()`, `resolve()`, and `validate_against(substrate)`. The binding from material name to executable lives here.
- The **re-expressed Activities Management bundle** in `bundles/activities_management.py`, now a thin selection of pool IDs rather than inline content.

The five pools and the registry continue to be extended by almost every later step; the bundle-as-selection shape persists unchanged.

What step 2 leaves for later

The substrate now holds the pools and the registry, and a bundle is a selection over both. What it does not yet do is *project* — assemble a runnable, in-memory practice from a bundle by resolving its pool references into a single object the practice server can hand to a harness. That is step 3's job. Step 4 then puts the projected practice behind an MCP surface.

Step 3: Projection

After step 2, anyone who wants to use a bundle has to walk three things at once: the bundle (for the id selections), the substrate (for the pool content), and the registry (for the bound callables). That is workable — step 2's `__main__.py` does exactly that — but it pushes the resolution rules onto every consumer. Step 3 collapses that walk into a single transformation.

What projection takes and produces

Projection is a function over three inputs:

```
project(bundle, substrate, registry) -> ProjectedPractice
```

The output is a self-contained object with all of the bundle's selections resolved inline and the material→callable bindings attached:

```
ProjectedPractice
  id, name, description          (from the bundle)
  ---
  teleo_affective : tuple of PoolElement      (resolved from
teleo_affective_ids)
  understanding   : tuple of PoolElement      (resolved from
understanding_ids)
  rules           : tuple of PoolElement      (resolved from rules_ids)
  affordances     : tuple of Affordance       (resolved from
affordance_ids)
  materials       : tuple of Material         (derived from the
affordances)
  bindings        : dict of name -> Callable (from the registry,
snapshot)
```

The projection function also validates: every id in the bundle must resolve into its pool, and every material the affordances reach for must have a binding in the registry. Validation runs once, at projection time, so consumers downstream do not need to re-check.

Projection as a snapshot

A projected practice is frozen for its lifetime. The pools it resolved against, the bindings it captured — all are taken at the moment of projection. If the substrate is later amended (a Smoother adds a rule in step 10, a dynamic material is registered at runtime), the projected practice already in use does not shift. A fresh projection picks up the change.

This matters because it gives a stable unit of "what the practitioner is engaged with right now". An LLM mid-enactment should not see the practice's content move under its feet because something elsewhere amended the substrate. Projection is the unit of stability.

Invoking through a projected practice

Because the projected practice carries both the descriptive surface and the bound callables, it can resolve an affordance call end-to-end on its own:

```
practice.invoke(affordance_id, material_name, arguments)
→ looks up the affordance in this practice's affordances
→ confirms material_name is one of that affordance's materials
→ looks up the callable in this practice's bindings
→ calls it with the arguments and returns the result
```

The consumer no longer needs to know about the substrate or the registry. The projected practice contains everything needed to enact it.

Activities Management projected

With the bundle from step 2, projection looks like this:

```
practice = project(ACTIVITIES_MANAGEMENT, substrate, FUNCTIONS)

practice.id == "activities_management"
len(practice.teleo_affective) == 1
len(practice.rules) == 4
len(practice.affordances) == 4
len(practice.materials) == 4 # derived from the affordances
len(practice.bindings) == 4 # one per material, snapshotted
from registry

practice.invoke(
    affordance_id="recent_activity",
    material_name="garmin_list_activities",
    arguments={"start_date": week_ago, "end_date": today},
)
# returns the list of synthetic activities for the period
```

Same bundle, same substrate, same registry as step 2 — just folded into one object that holds them all.

What step 3 contributes

Step 3 establishes the **projection** in [src/practice_theory_implementation/projection.py](#):

- The **ProjectedPractice** frozen dataclass, carrying the bundle's resolved content and the registry's bound callables in one self-contained object.

- The `project(bundle, substrate, registry)` function that does the bundle-against-substrate validation, derives materials from affordances, snapshots registry bindings, and returns the `ProjectedPractice`.
- `ProjectedPractice.invoke(affordance_id, material_name, arguments)` — end-to-end resolution against the projection's own data, no further substrate or registry lookup needed.

Projection is extended at step 6 to accept an `engagement` parameter and merge it additively into the result.

What step 3 leaves for later

The projected practice is now a self-contained, invocable object. What it is not yet is *reachable from outside the Python process*. Step 4 wires it behind an MCP server so a harness LLM can engage it through the standard MCP tool surface. Step 5 then introduces sessions and the enactment trail — recorded steps every time an affordance is invoked through the projected practice, so trust is enacted structure and not just messaging.

Step 4: The practice server

A projected practice is now a self-contained object inside the Python process. Step 4 puts it on a wire so a harness LLM can engage it.

The transport is MCP — the Model Context Protocol that Claude Code, Codex, Cursor and similar harnesses speak. An MCP server exposes a fixed set of tools and (optionally) resources; the harness connects, lists the tools, and calls them as part of its conversation with the user.

A note on the name. The prior essays called this layer the *apprenticeship server*, because in the full picture it is the server through which the harness LLM is apprenticed to the user. At this stage of the build the server only apprentices the LLM **in a practice** — it presents Activities Management and lets the harness invoke its affordances. It does not yet apprentice the LLM **about the user**: the standing teleo-affective, the rules of relation, the about-the-user knowing that hold across whichever practice is reached for. That is the apprenticeship *layer*, and it arrives in step 6 as a single move that wraps the engagement bundle into every projection. Once that lands, the same server becomes the apprenticeship server in full. For now, the more honest name is the *practice server*.

A fixed tool surface with dynamic affordances

There is a structural choice to make first: should each affordance in the active practice show up as its own MCP tool, or should the MCP surface stay fixed while the affordances surface through it as data?

The straightforward mapping would expose affordances directly as MCP tools — switching practice would change the tool list, and the harness would see the new affordances as new tools. This is what an MCP-native design would do.

The problem with that is portability. MCP clients vary in how well they handle `tools/list_changed` notifications; some clients ignore them, some re-list lazily, some refuse to refresh until the session reconnects. A surface that changes shape mid-session works against the lowest-common-denominator client.

The practice server takes a different approach: **the tool surface stays fixed, and affordances are reached through it as data**. At this step, five tools are exposed and never change for the life of the

session:

- **list_practices** — names every practice bundle in the catalog (id, name, description).
- **switch_practice(practice_id)** — replaces the session's active practice; the bundle is projected against the substrate and registry and the resulting ProjectedPractice becomes what the rest of the session works against.
- **current_practice** — returns the currently active practice: `{mode, practice: {id, name, description} | null, enactment_id, composition}`. The **composition** is the active practice's full projection rendered as markdown (engagement content merged in once Step 6 lands the engagement layer). Null when no practice has been switched into.
- **discover_affordances(query?)** — returns the active practice's affordances, optionally filtered by a query string against name and description. This is the dynamic surface — what affordances are available depends on which practice is active. Each affordance result carries its materials inline as `{name, description, input_schema}` objects (not just material names), so the harness LLM sees the argument shape it needs for **invoke_affordance** at the same moment it learns the affordance exists. The schema is on the material; surfacing it here closes the gap that a fixed generic tool surface would otherwise leave for per-material schemas.
- **invoke_affordance(affordance_id, material_name, arguments)** — dispatches through the active ProjectedPractice's **invoke()** and returns the result.

The tool list never changes (once registered). Affordances change through **discover_affordances**. Every MCP client knows how to call a tool, so the changing surface is reachable on every client without depending on notifications.

A sixth tool — **user_engagement** — is added in Step 6 alongside the engagement layer, registered only in somatic mode. The autonomic surface stays at five. That asymmetry between somatic (six tools by the end of Step 6) and autonomic (five) is honest: engagement is a somatic concept; the autonomic loop has no user to be engaged with. At this step, the surface is five tools for both modes; Step 8's mode split is what makes the asymmetry visible.

Active practice per session

Each MCP session holds one active practice at a time. The session opens with no practice active; the harness reaches for one via **switch_practice**. When **switch_practice** is called again, the previous active practice is discarded and the new one is projected fresh.

Source caveat: as written, the server holds the active practice in module-level globals rather than per-session state. Under stdio (one client per process) and a single HTTP client this is invisible. Concurrent HTTP clients would race on the shared active-practice slot — making the per-session model an intended shape rather than a present property until per-session lifespan state lands.

Switching is also the projection point. **switch_practice(practice_id)** does:

```
bundle = BUNDLES[practice_id]
practice = project(bundle, substrate, FUNCTIONS)
session.active_practice = practice
```

So the projection rules from step 3 — validation runs once, the snapshot is taken at this moment — apply each time a practice is switched into. Every active practice is a fresh snapshot.

Resolving an invocation

When the harness calls `invoke_affordance(affordance_id="recent_activity", material_name="garmin_list_activities", arguments={...})`, the server:

1. Looks up the session's active practice.
2. Calls `practice.invoke(affordance_id=..., material_name=..., arguments=...)`.
3. The projected practice resolves the affordance, confirms the material is in its list, looks up the bound callable, invokes it, and returns the result.
4. The server returns the result to the harness.

The server does not need to know about the substrate or the registry. The active practice carries everything.

Server instructions and a `practice://*` resource surface

Two MCP affordances sit alongside the tools and need brief mention here.

The first is **server-level instructions**, passed to FastMCP at construction. Every harness that connects sees them on `initialize`: a short brief on what this server is (the apprenticeship server) and where to start. At this step the instructions point the harness at `list_practices` as the entry point — there is no engagement layer yet, no user-focus brief to inherit. Step 6 lands the engagement bundle, and at that point the somatic instructions are updated to direct the harness to `user_engagement` before `list_practices` so the apprenticeship is read as a first-class thing before any practice is engaged; the autonomic instructions stay at `list_practices` since there is no engagement in autonomic mode. The instructions are not the bundle's composition — they are the *meta* the harness needs before it has switched a practice in. Bundle composition still flows through `current_practice` and the resources below.

The second is a **fixed resource surface**. Five `practice://*` resources are registered once and never change; the *content* of each changes whenever a practice is switched in:

- `practice://current` — the active projection's full composition as Markdown (engagement merged in, in somatic mode).
- `practice://teleo-affective`, `practice://understanding`, `practice://rules`, `practice://affordances` — the same composition split into its constitutive sections, one per resource, so a harness or auditor can read just the part it wants.

The resource surface is an alternative read-path to the inline `composition` field on `current_practice`. Clients that prefer the MCP resource model (templated URLs, content-type metadata, the option to subscribe) get a clean read; clients that prefer a single tool round-trip still get the same content through `current_practice`. Both paths render through `compose_composition()`, so the two views never disagree.

Transport

The server supports two transports, chosen at startup via `PRACTICE_TRANSPORT`:

- **stdio** (default) — the client launches the server as a subprocess and the two communicate over standard input and output. Each connection is its own process. Fits the verify, fits Codex's **.mcp.json** (Codex spawns the server per **codex exec**), and fits any harness that's happy to manage subprocesses.
- **http** — the server binds to a port (**PRACTICE_HTTP_HOST** / **PRACTICE_HTTP_PORT**, default **127.0.0.1:7180**) and runs as a long-lived process. Aimed at production autonomic deployment where Anthropic-SDK workers and the user's harness connect to one substrate. Genuine concurrent use is gated on the per-session-state work noted above; until then the safe pattern is one HTTP client per server.

Both transports run the same tool surface (six in somatic, five in autonomic — see step 6 for the asymmetry), the same projection, the same dispatcher. The MCP machinery is transport-agnostic; switching is a configuration choice, not a change to the tools.

```
# stdio (default) – for verify, Codex, anything that wants a subprocess
uv run python -m practice_theory_implementation.server

# HTTP – long-lived server; one client per server until per-session state
lands
PRACTICE_TRANSPORT=http PRACTICE_HTTP_PORT=7180 \
  uv run python -m practice_theory_implementation.server
```

A repo-root **.mcp.json** declares **practice_server_somatic** as a stdio entry so Codex (and other clients that read **.mcp.json**) can spawn the somatic surface with one command. The autonomic server is deliberately *not* registered there: user-facing harnesses connecting via **.mcp.json** should not see the autonomic surface (exposing **judge_emit_friction** to the user seat is a footgun), and the autonomic runner reaches its server through other paths — **CodexExecAdapter** injects the autonomic config inline via **codex exec -c mcp_servers...**, **ClaudeCliAdapter** via **--mcp-config**, **AnthropicSDKAdapter** via a spawned stdio subprocess or an HTTP URL. For a long-lived autonomic HTTP server, start it directly: **PRACTICE_TRANSPORT=http PRACTICE_SERVER_MODE=autonomic uv run python -m practice_theory_implementation.server**.

Activities Management on the wire

With one bundle in the catalog (**activities_management**) the session looks like this when exercised:

```
> list_practices()
[
  { "id": "activities_management",
    "name": "Activities Management",
    "description": "Keep an honest, useful view of..." }
]

> switch_practice("activities_management")
{ "active": "activities_management" }

> current_practice()
{ "mode": "somatic",
```



```

    "practice": { "id": "activities_management", "name": "...",
                  "description": "..." },
    "enactment_id": "a7c1...",
    "composition": "# Activities Management\n\n## Teleo-affective\n..." }

> discover_affordances()
[
  { "id": "recent_activity", "name": "Recent activity",
    "description": "Look at activities over a recent window...",
    "materials": [
      { "name": "garmin_list_activities",
        "description": "List the user's activities within a date
range.",
        "input_schema": { "type": "object",
                          "properties": {
                            "start_date": { "type": "string", "format": "date" },
                            "end_date": { "type": "string", "format": "date" },
                            "activity_type": { "type": "string" } },
                            "required": ["start_date", "end_date"] } }
    ] },
    ...
]

> invoke_affordance(
  affordance_id="recent_activity",
  material_name="garmin_list_activities",
  arguments={ "start_date": "2026-05-19", "end_date": "2026-05-25" })
[
  { "date": "2026-05-19", "type": "cycle", ... },
  ...
]
```

The catalog has one bundle for now. Step 6 (the apprenticeship layer) will add the standing engagement bundle that wraps practices, and from then on the catalog has more than one and `switch_practice` does real switching.

What step 4 contributes

Step 4 establishes the practice server:

- `src/practice_theory_implementation/server.py` — the FastMCP `mcp_app` with the five-tool surface (`list_practices`, `switch_practice`, `current_practice`, `discover_affordances`, `invoke_affordance`), a mode-aware `instructions=` brief surfaced to harnesses on `initialize`, and a fixed `practice://*` resource surface (`current`, `teleo-affective`, `understanding`, `rules`, `affordances`) reading from the active projection. Supports stdio and HTTP transports via `PRACTICE_TRANSPORT`.
- `src/practice_theory_implementation/bundles/__init__.py` — the `BUNDLES: dict[str, Bundle]` catalog the server lists from. Holds Activities Management initially; later steps add more.
- The MCP dependency in `pyproject.toml` (`mcp>=1.2.0`), the JSON-friendly date coercion in the Garmin mocks, and the verify in `__main__.py` rewired to subprocess the server and drive it over a

real MCP client session.

The five-tool surface gains **user_engagement** at step 6 (somatic only); the catalog grows and gains the autonomic / somatic mode filter at step 8.

What step 4 leaves for later

The practice server now exposes the projected practice through MCP, but every invocation is fire-and-forget — call it, get a result, nothing recorded. Step 5 introduces **engagement** and **enactment**: each session opens an engagement around the active practice, each **invoke_affordance** call records a step on an enactment trail, and the trail becomes the inspectable record of what was done with the practice. Trust as enacted structure begins there.

Step 5: Engagement and enactment

After step 4 the harness can drive a practice end-to-end through the MCP surface, but nothing is left behind. Each call is fire-and-forget — invoke, get a result, move on. There is no record of what happened, which means there is nothing for a later reader (a Judge in step 9, the user themselves, an audit) to look at. Trust at this point rests entirely on what the LLM says it did, not on what was actually done.

Step 5 fixes that. Each call becomes a recorded step on an inspectable trail.

Two terms, distinct

Engagement is the standing relationship a session has with the server. It opens when the session connects and closes when the session disconnects. It is the "we are working together right now" relation. At step 5 the engagement is implicit — the session itself is the engagement, and we do not yet persist a separate record for it. Step 6's apprenticeship layer will give the engagement its own bundle and make it materially present in the trail.

Enactment is a discrete period of doing-a-practice within an engagement. It opens when a practice is switched into and closes when the practice is switched away from or the session ends. A single engagement can contain many enactments — one per practice the session reaches for. Each enactment is recorded.

Step 5 is about enactments and the steps they hold.

The trail

The trail lives in SQLite. Two tables, kept small:

```
enactments
  id          : text          (unique)
  practice_id : text
  opened_at   : text          (ISO timestamp)
  closed_at   : text          (ISO timestamp, NULL while open)

steps
  id          : integer (auto)
  enactment_id : text      (references enactments.id)
```

```

affordance_id    : text
material_name    : text
arguments_json   : text    (the arguments passed, JSON-encoded)
result_summary   : text    (the result, JSON-encoded and truncated)
started_at       : text    (ISO timestamp)
completed_at     : text    (ISO timestamp)
duration_ms      : integer (monotonic)

```

Every **invoke_affordance** call writes a steps row. Every **switch_practice** call closes the current enactment (if any) and opens a new one. The schema is small on purpose — step 5's job is to make the trail exist and be readable, not to add columns for every possible future need.

Trail as enacted structure

The point of the trail is that trust no longer rests on what the LLM says. The trail records what was actually invoked, with what arguments, against what practice, and what came back. A reader — the user, a Judge in step 9, an auditor at some later date — can examine the trail directly. If the practice's rules forbid prescriptive coaching and the trail shows nothing but data lookups, the rule is being honoured at the surface where it matters. If the practice's affordances were never reached for and the LLM was free-styling in prose instead, the trail makes that absence visible.

This is what the prior essays meant by *trust as enacted structure*. The structure is the trail. Without it, trust is messaging — the LLM tells the user it did the right thing and the user has to take its word. With it, the doing is inspectable.

Wiring the server

The trail integrates into the practice server at two points:

- **switch_practice(practice_id)** — closes the previous enactment (writes **closed_at**), opens a new one (insert a row with **opened_at**), and sets the new active practice. The session's "current enactment" updates to point at the new row.
- **invoke_affordance(affordance_id, material_name, arguments)** — wraps the dispatch through the projected practice with timing, then writes a steps row for the current enactment with the arguments, result, and timing.

Nothing in the projection or the registry needs to know about the trail; the wiring lives at the server layer, around the calls into the projected practice.

Activities Management with a trail

The verify script from step 4 makes the same calls but now leaves a record. After the MCP session closes, opening **data/trail.db** shows something like:

```

Enactment a7c1...  practice=activities_management  opened 2026-05-25T...
closed (open)
  Step 1  affordance=recent_activity  material=garmin_list_activities
          arguments={"start_date": "2026-05-19", "end_date": "2026-05-
25"}

```

```

    result=[{"activity_id": "act-2026-05-19-482", ...}, ...]
(truncated)
    duration_ms=0

```

For longer sessions the trail accumulates — one enactment per practice switched into, one step per affordance invoked. Step 9's Judge reads exactly this surface when it looks at what an enactment did.

What step 5 contributes

Step 5 establishes the enactment trail:

- `src/practice_theory_implementation/trail.py` — `EnactmentStore` wrapping SQLite at `data/trail.db`, with `open_enactment`, `close_enactment`, `record_step`, `recent_enactments`, `steps_for`, and a `time_call()` context manager for ground-truth timing.
- The `enactments` and `steps` tables (the schema is added in `EnactmentStore.__init__`).
- The wiring in `server.py`: `switch_practice` closes the previous enactment and opens a new one; `invoke_affordance` writes a steps row for every call, including failures.

The trail's `enactments` table gains a `parent_enactment_id` column at step 6, and a `friction_observations` table at step 9. Step 11 adds the `judge_inbox` and `smoother_inbox` tables to the same database.

What step 5 leaves for later

The trail is present and readable but nothing yet acts on it. Step 6 wraps the engagement bundle into the projection so the apprenticeship layer becomes material (and the trail starts carrying engagement-level steps alongside practice-level ones). Step 7 introduces Practice Management, which gives the substrate a mutable overlay so amendments can land at runtime. Step 9 introduces the Judge — an autonomic practitioner that reads closed enactments and emits Friction observations. Step 10 introduces the Smoother — another autonomic practitioner that amends the substrate in response, reusing Practice Management's meta-materials. The trail is the substrate they read from.

Step 6: The apprenticeship layer

Up to here, every step has built infrastructure. Step 6 changes what the server actually *does*. The practice server gains a standing arrangement with the user — the apprenticeship layer the prior essays named — and earns its proper name.

Engagement is not a practice

The first move is conceptual, not technical. **Engagement** is the standing relationship between the LLM and this user; **practices** are the discrete things done within that relationship. Activities Management is a practice; the relationship that holds whether the user is looking at activities, drafting correspondence, or just sitting with the harness in silence is not. Doc 2 called this distinction explicit:

The standing relationship between the LLM and the user — the one that holds across whichever practice is being done at the moment — is not a thing one does. It is the container within which practices are reached for. The LLM does not enact this arrangement; the LLM is *in* it.

The implementation respects that distinction. Engagement is structurally separate from practice — it does not appear in `list_practices`, it cannot be switched to, and the harness does not engage it explicitly. It is projected once when the session opens and is present beneath every practice the session reaches for.

The engagement bundle

The engagement is expressed in the same five-element shape a practice is — same `Bundle` dataclass, same pool decomposition. The ontological distinction is in the bundle's role, not its structure. A single engagement bundle (`user_focused_engagement`) lives in the codebase alongside the practice bundles but in a structurally separate slot.

What it carries:

- **Teleo-affective** — the standing posture: be here for this user; attend to what they bring; the user is sovereign.
- **Understanding** — about-the-user knowing: who this is, where they are, what they care about right now. At step 6 this is a single inline content; later steps can back it with a richer store.
- **Rules** — the disciplines of relation that hold across whichever practice is active: do not displace what the user brings, offer rather than instruct, honour what they have brought.
- **Affordances** — the engagement-layer moves: at minimum `about_the_user` (consult what is known about this user). These are *always* available, in every projected practice, regardless of which practice is switched in.
- **Materials** — what those affordances reach for. At step 6 a single mock material backs `about_the_user`.

Additive merge into every projection

Step 3's `project()` is updated to take an optional `engagement` parameter. When provided, the engagement's content is merged additively into the result:

```
project(bundle, substrate, registry, engagement=eng) -> ProjectedPractice
  teleo_affective = eng.teleo_affective + bundle's teleo_affective
  (deduped)
  understanding   = eng.understanding   + bundle's understanding
  (deduped)
  rules          = eng.rules             + bundle's rules
  (deduped)
  affordances     = eng.affordances      + bundle's affordances
  (deduped)
  materials       = derived from the merged affordances
  bindings        = snapshot for all merged materials
```

A practice no longer carries the engagement's content in its own bundle (no duplication across practices); the projection assembles the merged view at projection time. From the harness's perspective, the projected practice is a single coherent object — the engagement's posture, the engagement's rules, the engagement's affordances, plus the practice's own. The split is invisible at the surface.

The trail materialises the layering

The trail's **enactments** table gains a **parent_enactment_id** column. When the session opens:

1. The engagement is projected. An **engagement enactment** is opened — practice id is the engagement bundle's id, parent_enactment_id is NULL.
2. When **switch_practice(practice_id)** is called, the practice is projected (with the engagement folded in) and a **practice enactment** is opened — practice id is the practice's id, parent_enactment_id points at the engagement enactment.
3. Every **invoke_affordance** call records a step against the enactment that owns the affordance. Engagement-layer affordances (like **about_the_user**) record against the engagement enactment; practice-layer affordances (like **recent_activity**) record against the practice enactment.

The trail now shows the layering explicitly. A reader can see "this session held an engagement; within it the user reached for these practices; within each practice these steps happened". The structure of the apprenticeship is in the data.

A second practice in the catalog

So far the catalog has held only Activities Management, which made **switch_practice** something to demonstrate but not really to *use*. Step 6 adds a second small practice — **Reflection** — so the apprenticeship's "holds across practices" point becomes visible. The user can switch from Activities Management to Reflection and the engagement's content (the about-the-user knowing, the standing rules) carries unchanged. Each switch closes the previous practice enactment and opens a new one; both link to the same engagement enactment.

Reflection itself is tiny:

- one affordance — **record_reflection**, which stores a short written reflection
- one mock material — **store_reflection**, which echoes the text and returns a stub id

Enough to make switching real without inflating the example.

The server, now apprenticing

The practice server from step 4 changes in one place: at server startup it projects the engagement bundle and opens an engagement enactment that lives for the whole process. Every **switch_practice** call passes the engagement into **project()** so the active practice carries the merged content. Every **invoke_affordance** call dispatches through the active practice's **invoke()** and records the step against whichever enactment the affordance belongs to.

The somatic tool surface gains one tool when the engagement layer arrives: **user_engagement**, alongside the five from step 4. **user_engagement** returns the engagement bundle's content (id, name, description, composition) without any practice merged in — it makes the apprenticeship layer a first-class read, independent of which practice is currently engaged. The other tools are unchanged in shape from step 4, but **current_practice** now returns the active practice's *full projection rendered as markdown* (engagement content merged in) in its **composition** field, so the harness can see exactly what an LLM enacting the active practice sees.

The autonomic surface stays at five — engagement is a somatic concept, and the autonomic loop has no user to be engaged with. The asymmetry between the two modes is honest and intentional.

What is different now is what the projection contains: the harness LLM is apprenticed *about* the user (through the engagement's understanding and rules) as well as *in* the active practice (through the practice's own content). The server has earned the name the prior essays gave it: **the apprenticeship server**.

What step 6 contributes

Step 6 establishes the apprenticeship layer:

- The **engagement bundle** in `src/practice_theory_implementation/bundles/user_focused_engagement.py`, in a structurally separate slot from the practice catalog (`ENGAGEMENT_BUNDLE`, deliberately not in `BUNDLES`).
- The **about_the_user** affordance and **consult_about_user** material behind it (mock in `materials/about_user_mock.py`).
- A **second practice in the catalog**, Reflection — in `bundles/reflection.py` with one affordance backed by `materials/reflection_mock.py` — so switching between practices is real rather than demonstrative.
- The **additive merge in projection**: `project(bundle, substrate, registry, engagement=eng)` folds the engagement's content into the result. Engagement first, deduped by id; materials and bindings derive from the merged affordance set.
- The **parent_enactment_id** column on `enactments`, and the server logic that opens the engagement enactment at startup and links each practice enactment to it.
- The **sixth tool** on the somatic surface — `user_engagement` — making the engagement bundle a first-class read.

The engagement projection becomes mode-aware at step 8 (somatic-only).

What step 6 leaves for later

The apprenticeship layer is present and the trail records both engagement and practice enactments, but the substrate is still a Python module — runtime amendments require a restart. Step 7 introduces **Practice Management**, the somatic meta-practice for authoring and amending the substrate at runtime: a mutable overlay on top of the seed pools, meta-materials (`create_element`, `amend_element`, `create_bundle`, `amend_bundle`, and so on), and the Practice Management bundle whose affordances reach for them. Step 8 introduces the **somatic/autonomic** distinction — splitting the server's surface so practices that need the user are kept distinct from practices that can act alone. Step 9 introduces the **Judge** — an autonomic practitioner that reads closed enactments (engagement and practice both) and emits Friction observations. Step 10 introduces the **Smoother** — another autonomic practitioner that amends the substrate in response, reusing Practice Management's meta-materials. Together with Practice Management itself they close the strange loop the second essay described.

Step 7: Practice Management

The substrate so far has been frozen at startup. The seed pools in `pools.py` are read once, the catalog of bundles is built from imported modules, and nothing changes until the process restarts. That is fine for steps 1 to 6 — the apparatus needs to exist before it needs to be amendable — but it is incompatible with

what doc 2 named: a practice for *authoring practices*, enacted while the system is running, on the user's behalf.

Step 7 makes the substrate mutable, introduces the meta-materials that mutate it, and adds the Practice Management bundle that exposes those meta-materials as affordances.

Seed and overlay

The substrate becomes layered:

```
seed pools (pools.py – immutable Python)
+ overlay (data/substrate.db – mutable SQLite)
= effective substrate (loaded at server startup; mutated at runtime)
```

The seed is the floor — what the system starts with, version-controlled, reviewable. The overlay is where runtime amendments land. When the server starts, it reads both and merges them into the in-memory **Substrate** (overlay wins on id collision). The same merge applies to the bundle catalog — seed bundles plus overlay bundles.

When a meta-material runs, it writes to the overlay (so the change survives a restart) and updates the in-memory substrate (so the change is visible to subsequent projections immediately). Existing projections do not shift — projection is still a snapshot, as step 3 established. A new projection picks up the amendment.

The meta-materials

Practice Management exposes nine meta-materials. They divide cleanly along the substrate's shape:

```
pm_read_pool(pool)                – inspect a pool's contents
pm_create_element(pool, id, name, content) – add a teleo_affective /
understanding / rules entry
pm_amend_element(pool, id, ...)    – change one
pm_create_affordance(id, name, description, materials) – add an
affordance
pm_amend_affordance(id, ...)       – change one
pm_create_material(name, description, input_schema) – add a material's
captured surface
pm_amend_material(name, ...)       – change one
pm_create_bundle(id, name, description, ids...) – add a bundle (a
selection over the pools)
pm_amend_bundle(id, ...)           – change one
```

Each **create** validates that ids do not collide and that referenced ids resolve into their pools. Each **amend** is partial — only the fields the caller provides are changed. The **read_pool** material lets the LLM see what is already there before reaching for **create** or **amend**.

A note on **pm_create_material**. The material's captured surface (name, description, input schema) is what gets stored. The executable behind the name — the callable in the function registry — is not created by **pm_create_material**. A new material's name must already correspond to a function registered in

`registry.py`, or projection of any bundle that uses it will fail. Step 7 keeps the captured/executable separation step 1 promised; runtime authoring of new executables (the dynamic-materials decorator path) is a later refinement.

The Practice Management bundle

The bundle itself is short. Its teleo-affective names the orientation (*author and amend the substrate on the user's behalf, with the user in the loop*); its understanding describes the substrate's shape (five pools plus a bundle catalog, last-write-wins, ids unique within pool); its rules carry the disciplines of substrate work (preview before applying, do not collide with existing ids, treat amendments as additions not replacements). Its affordances expose the nine meta-materials as high-level moves — *read pool*, *author pool element*, *amend pool element*, *author affordance*, and so on.

Practice Management is somatic. The user is in the loop because authoring a new practice requires deciding what the practice is for, what its teleo-affective should be, what rules apply — none of which the LLM should answer alone. Step 8 will make the somatic/autonomic distinction explicit; for step 7 it is enough that PM expects the user.

Authoring a practice at runtime

The verify script demonstrates Practice Management actually being used. It:

1. Switches to Practice Management.
2. Reads the rules pool to see what is already there.
3. Creates a new teleo-affective element, a new understanding element, and a new rule via `pm_create_element`.
4. Creates a new affordance (`pm_create_affordance`) that reaches for an existing Garmin material.
5. Creates a new bundle (`pm_create_bundle`) that selects the new elements and the new affordance.
6. Switches into the newly-authored bundle.
7. Invokes the bundle's new affordance.

Six of the seven steps record into the Practice Management enactment; the seventh records into the new bundle's enactment. The trail shows the layering as before: engagement at the top, Practice Management as one practice enactment under it, the newly-authored practice as another. The substrate after the run carries the new content; the next server start would load it from the overlay.

A strange-loop precursor

Practice Management is itself a practice — same Bundle shape, same five elements, same projection rules. That means Practice Management can be enacted *through* Practice Management: a Smoother in step 10 can use `pm_amend_bundle` to amend Practice Management's own affordances, or `pm_create_element` to add a new rule to Practice Management's rules pool. The machinery improves itself through the same machinery it was built with. Step 12 names this explicitly; here we just notice that step 7 puts the mechanism in place.

What step 7 contributes

Step 7 establishes Practice Management — the meta-practice for authoring practices at runtime:

- The **substrate overlay** in `src/practice_theory_implementation/substrate_store.py`: `SubstrateStore` over SQLite at `data/substrate.db`, with overlay tables for pool elements, affordances, materials, and bundles, plus the two merge functions that apply the overlay over the seed at startup.
- The **nine meta-materials** in `materials/practice_management.py` (`pm_read_pool`, `pm_create_element`, `pm_amend_element`, and parallel pairs for affordances, materials, and bundles).
- The **Practice Management bundle** in `bundles/practice_management.py`, exposing the meta-materials as nine authoring affordances. PM is somatic — the user is in the loop for authoring.
- Seed pool entries in `pools.py` for PM's teleo-affective, understanding, three rules, nine affordances, and nine materials.
- The server's startup-time wiring: open the store, apply the overlay, configure PM with references to substrate / catalog / store.

The Smoother at step 10 reuses six of PM's amendment affordances directly from the shared pool — the autonomic mirror of the same machinery.

What step 7 leaves for later

Practice Management is in the catalog and the substrate is mutable, but everything is still served through one surface — the harness has to keep track of which practices need the user and which can run alone. Step 8 splits the surface into **somatic** (user in the loop) and **autonomic** (acting alone) endpoints, so the autonomic loop in steps 9 and 10 has a place to live without colliding with what the user is doing.

Step 8: Somatic and autonomic modes

Doc 2 introduced two kinds of practice: **somatic** practices need the user — they require the user to state their mind — and **autonomic** practices act alone, with no human in the loop. Activities Management, Reflection, and Practice Management are all somatic; the Judge and Smoother coming in steps 9 and 10 are autonomic.

Both kinds run on the same machinery — same Bundle shape, same projection, same enactment trail — and they share the same substrate. What differs is *who* enacts them and *under what conditions*. Step 8 makes that difference structural so the autonomic loop has somewhere to live without stepping on the user's session.

Bundles carry a mode

Each bundle declares its mode:

```
Bundle
  id, name, description
  mode : "somatic" | "autonomic"
  ---
  teleo_affective_ids, understanding_ids, rules_ids, affordance_ids
```

All three existing practice bundles — Activities Management, Reflection, Practice Management — are marked **somatic**. The engagement bundle is not in the catalog (it cannot be switched to) so its mode is

moot; in practice it is only ever projected in a somatic session.

When the Judge and Smoother bundles arrive in steps 9 and 10, they will be marked **autonomic**.

A mode-aware server

The server gets a mode at startup. Two changes follow:

1. **The catalog the server exposes is filtered by mode.** `list_practices` returns only bundles whose `mode` matches the server's mode. `switch_practice` rejects an attempt to switch to a wrong-mode bundle.
2. **The engagement is projected only in somatic mode.** Autonomic practitioners (Judge, Smoother) are not "for the user" — they are about tending the substrate. They have no user-focus to inherit. In somatic mode the engagement is projected at startup as before; in autonomic mode it is not, and practice enactments are top-level (`parent_enactment_id` is NULL).

Everything else is unchanged. The same MCP tool surface (six in somatic where `user_engagement` is present, five in autonomic where it is not), the same projection rules, the same trail. The mode is the filter at the catalog edge, the switch on whether engagement gets projected, and the gate on the `user_engagement` tool registration.

Stdio with a mode flag

For the demonstration, the server's mode is set by an environment variable:

```
PRACTICE_SERVER_MODE=somatic    python -m
practice_theory_implementation.server
PRACTICE_SERVER_MODE=autonomic  python -m
practice_theory_implementation.server
```

Stdio is one process per client, so each mode is its own process. The verify script subprocesses the server twice in sequence — first in somatic mode (the existing walk-through and the Practice Management authoring), then in autonomic mode (which currently has no bundles to switch to, because Judge and Smoother arrive in the next two steps).

For a real deployment with the user's harness and the autonomic loop both running, the natural shape is **HTTP transport with two URL paths** — `/mcp/somatic/` and `/mcp/autonomic/` against the same server process. Both endpoints share the substrate, the registry, and the trail; only the catalog filter and the engagement projection differ. The transport change is a configuration concern at the server's entry point; the tools and bundles do not change.

What the autonomic session looks like at step 8

```
> list_practices()
[]                                # no autonomic bundles yet

> switch_practice('activities_management')
{ "error": "practice 'activities_management' is somatic; server is
```

```
autonomic" }

> current_practice()
{ "active": null, "mode": "autonomic" }
```

This is the cleanest possible answer at step 8 — the autonomic surface exists, it knows what it is, and it has nothing to enact until the autonomic practitioners arrive. Step 9 (Judge) and step 10 (Smoother) populate it.

What step 8 contributes

Step 8 establishes the somatic / autonomic split:

- A **mode field on Bundle** in `types.py` (`Mode = Literal["somatic", "autonomic"]`, defaulting to `"somatic"`); the three existing practice bundles each declare it explicitly.
- A **mode column on bundle_overlay** in `substrate_store.py`, so runtime-authored bundles round-trip their mode; `pm_create_bundle` is extended to accept it.
- A **mode-aware server**: `PRACTICE_SERVER_MODE` is read at startup; `list_practices` filters by mode; `switch_practice` rejects mismatched modes; the engagement is projected only in somatic mode (autonomic `_engagement` is `None`); `user_engagement` is registered only in somatic mode.
- The verify is refactored into `verify_somatic()` and `verify_autonomic()`, each spawning its own server subprocess with the appropriate mode env var.

The autonomic surface has nothing to switch to until step 9 (Judge) and step 10 (Smoother) populate it.

What step 8 leaves for later

The autonomic surface is in place but empty. Step 9 introduces the **Judge**: an autonomic bundle with primitives for reading the trail and emitting **Friction** observations. Step 10 introduces the **Smoother**: an autonomic bundle with primitives for reading Friction and marking it addressed, reusing Practice Management's amendment affordances to actually change the substrate. Step 11 wires both into an **autonomic harness** that drives real LLM enactment, with adapters spanning two providers (Anthropic, Codex) and two process shapes (long-lived in-process, subprocess per dispatch) behind a single abstraction. Step 12 then names the **strange loop**.

Step 9: The Judge

The trail has been recorded since step 5; nothing has read it. The Judge is the first practitioner whose work *is* reading the trail. It is an autonomic bundle — it acts without the user — that looks at recent enactments and emits **Friction** observations about what happened.

What a Friction is

A Friction is a small structured record that says "this thing in the trail is worth attending to". Its fields:

```
FrictionObservation
  id                (auto)
  observing_enactment_id  the Judge enactment that emitted it
  target_enactment_id    the enactment being judged
  kind                a short label (e.g., "narrow_engagement")
```

content	freeform description of what the Judge saw
observation_data	optional structured evidence (what was observed)
observed_at	when the Judge emitted this
addressed_at	NULL until a Smoother addresses it
addressed_by_enactment_id	which Smoother enactment addressed it

Frictions live in their own table in the trail database. They are **observations, not commands**. The Judge does not propose remedies; **observation_data** is *evidence* — e.g., for **narrow_engagement**, the used and unused affordance sets — and the Smoother decides what (if anything) to do.

The Judge's primitives

The Judge's intelligence does not live in code. It lives in the Judge bundle's understanding and rules — prose the enacting LLM reads to know what kinds of friction to look for. The materials underneath are four small primitives the LLM composes during an enactment:

- **list_recent_enactments(limit, bundle_id?)** — what is there to judge.
- **read_enactment_steps(enactment_id)** — what an enactment did.
- **read_bundle(bundle_id)** — what the enactment had available, by way of comparison.
- **emit_friction(target_enactment_id, kind, content, observation_data?)** — record an observation.

No heuristic is baked into the materials. A Judge enactment reads, compares, and decides what to name. In production, that deciding is the LLM's enactment of the Judge bundle. The verify (for deterministic demonstration without an LLM) walks the same four primitives in sequence and makes a simple **narrow_engagement** choice in Python — explicitly playing the role the LLM would play. Step 11's autonomic harness replaces the verify-as-scripted-LLM with a real LLM enactment.

The Judge bundle

- **Mode:** autonomic.
- **Teleo-affective:** read the trail and call out what is worth attending to; the Judge serves the practitioner who comes after, not the user directly.
- **Understanding:** the prose carries the heuristics — what to compare across enactment and bundle, what kinds of friction to look for (**narrow_engagement**, **rule_neglect**, **repetition** are starting points), what **content** and **observation_data** should carry.
- **Rules:** examine before naming; one thing per friction; observe, do not remediate.
- **Affordances:** **list_recent_enactments**, **read_enactment_steps**, **read_bundle**, **emit_friction** — the four primitives.

The Judge's enactment itself is recorded in the trail like any other enactment. Step 12 (the strange loop) makes use of this: the Judge's enactments can be judged.

What step 9 contributes

Step 9 establishes the Judge:

- The **friction_observations table** in `trail.py`, with `FrictionRow` and the `EnactmentStore` methods `record_friction`, `pending_friction`, `all_friction`, `mark_friction_addressed`, `most_recent_enactment_of(practice_id)`.
- The **four primitives** in `materials/judge.py`: `judge_list_recent_enactments`, `judge_read_enactment_steps`, `judge_read_bundle`, `judge_emit_friction`. No heuristic logic — the LLM enacting the bundle decides what to name.
- The **Judge bundle** (autonomic) in `bundles/judge.py`, with the four primitive affordances and the heuristics-in-prose understanding.
- Seed pool entries in `pools.py` for the Judge's teleo-affective, understanding, three rules, four affordances, four materials.

The Judge's own enactments are recorded in the trail like any other; step 12 observes that they can be judged in turn.

What step 9 leaves for later

Friction observations now exist in the trail but nothing acts on them. Step 10 introduces the **Smoother** — the autonomic counterpart that reads pending Friction, amends the substrate to address it, and marks the Friction addressed.

Step 10: The Smoother

The Smoother is the second autonomic practitioner. Where the Judge observes and emits Friction, the Smoother takes a Friction and *acts on it* — by amending the substrate. It does this through the meta-materials Practice Management introduced in step 7. That reuse is the conceptual move step 10 makes visible.

Reusing Practice Management's affordances

This is the move step 10 makes structurally visible: **the Smoother bundle references Practice Management's amendment affordances directly**. The substrate's affordance pool is shared; bundles select from it. PM and Smoother select overlapping sets — same `amend_pool_element`, `amend_affordance`, `amend_material`, `amend_bundle`, `author_pool_element`, `read_pool` affordances appear in both. The bundles differ in what they hold around those affordances (teleo-affective, rules, the surrounding framing); the amendment machinery underneath is one machinery.

This is the autonomic-somatic split: same materials, two enacted contexts. When the user enacts Practice Management, they reach for `amend_bundle` through PM's framing. When the Smoother enacts autonomically, it reaches for the same `amend_bundle` through its own framing. No duplication; the pool layer makes the reuse cheap.

Smoother's primitives

The Smoother's own materials are only two:

- `smoother_read_pending_friction(limit)` — returns the unaddressed Friction.
- `smoother_mark_addressed(friction_id)` — marks a Friction addressed by the active Smoother enactment.

The amendment work itself uses Practice Management's `pm_*` materials, reached for through the reused affordances. The Smoother has no switch-on-kind logic and no per-Friction handler. Interpretation lives in the bundle's understanding (prose).

The Smoother bundle

- **Mode:** autonomic.
- **Teleo-affective:** read pending Friction, interpret what was named, apply the smallest substrate amendment that addresses it, mark addressed.
- **Understanding:** prose that names kinds (`narrow_engagement`, `rule_neglect`, `repetition`) and what kinds of amendment fit each. Starting points only — the enacting LLM may amend differently when the Friction calls for it.
- **Rules:** address only what the Friction names; do not invent Friction; mark addressed when done.
- **Affordances:** two smoother-specific (`read_pending_friction`, `mark_friction_addressed`) plus six reused from Practice Management (`read_pool`, `amend_pool_element`, `author_pool_element`, `amend_affordance`, `amend_material`, `amend_bundle`).

The Smoother's enactment is recorded too. When it completes an amendment, the substrate carries the change, and any new projection — including the user's next somatic session — picks it up.

The full loop end-to-end

The verify script runs three phases in sequence:

1. **Somatic** — the existing walk (engagement, switch to Activities Management, invoke one of its affordances, switch to Reflection, ..., Practice Management authoring Quick Glance). Closes practice enactments populated with steps.
2. **Autonomic Judge** — switches to the Judge bundle and walks the four primitives in order: list recent enactments, read one enactment's steps, read the corresponding bundle, emit a Friction describing what was observed (with `observation_data` carrying the evidence). The verify makes the `narrow_engagement` decision in Python — the same decision an LLM enactment would make from reading the Judge bundle's understanding.
3. **Autonomic Smoother** — switches to the Smoother bundle, reads pending Friction, interprets `observation_data`, applies an amendment through `amend_bundle` (PM's affordance), marks the Friction addressed. The verify makes the interpretation in Python — again, the same work an LLM enactment would do.

After the run the trail shows three top-level structures (somatic engagement with its practice enactments, Judge enactment, Smoother enactment), the `friction_observations` table holds one addressed Friction with `observation_data` (not a remedy), and `activities_management` in the catalog carries the amended description.

Step 11 (the autonomic harness) is what makes phases 2 and 3 happen with a real LLM rather than a scripted verify.

Trust at the autonomic edge

The Smoother acts without the user; it amends the substrate the user's own practitioners will be projected against next time. That is the highest-trust-cost move in the architecture — an autonomous process

changing the rules its principal operates under. The trail is how the move stays honest. Every Smoother enactment is recorded with the same structure as any other; the Friction it addressed carries `addressed_by_enactment_id` pointing back at the amending enactment; the amendment lands in the overlay, where it can be read alongside the seed. The user can examine, for any change that took effect: which Friction was named, by which Judge enactment, addressed by which Smoother enactment, with what amendment. Nothing about the autonomic move is hidden — the trust thesis from step 5 holds at the edge where it is hardest to hold.

What step 10 contributes

Step 10 establishes the Smoother:

- The **two Smoother primitives** in `materials/smoother.py`: `smoother_read_pending_friction(limit)` and `smoother_mark_addressed(friction_id)`. Amendment work is delegated to Practice Management's `pm_*` materials, not duplicated.
- The **Smoother bundle** (autonomic) in `bundles/smoother.py` with eight affordances — two Smoother-specific plus six **reused from Practice Management's pool** (`read_pool`, `amend_pool_element`, `author_pool_element`, `amend_affordance`, `amend_material`, `amend_bundle`). This is the autonomic-somatic mirror: same materials, two enacted contexts.
- Seed pool entries in `pools.py` for the Smoother's teleo-affective, understanding (heuristics-in-prose for interpreting Friction), three rules, two Smoother-specific affordances, two materials. The six reused PM affordances and materials are already in the pool from step 7 — no duplication.
- Server-side `configure_smoother(...)` alongside the Judge wiring.

Two honest notes carry forward to step 11:

- **The verify scripts the LLM's role.** The Judge's "what kind of friction is this" and the Smoother's "what amendment fits this Friction" are both done in Python in the verify, deterministically. In production an LLM enactment of each bundle reads the bundle's understanding and rules and does the same work. Step 11's autonomic harness wires the real LLM enactment in.
- **The amendment is not idempotent.** Re-running the verify against an already-amended substrate produces another Friction, and the Smoother appends the suggested text again. A real Judge enactment, reading the already-amended description, might or might not name a fresh Friction — that judgment is what LLM enactment brings.

What step 10 leaves for later

Steps 9 and 10 give the Judge and Smoother primitives the LLM needs to do its work, and the substrate machinery to record what gets done. What is still missing is **the LLM itself, driving the loop autonomically**. The verify above stands in by scripting deterministic choices in Python; a real deployment runs an LLM that reads the trail and decides each turn. Step 11 introduces the **autonomic harness** that wires that real LLM enactment in, with three real adapters (spanning Anthropic and Codex, in-process and subprocess) behind a single abstraction so the substrate need not know which provider or shape is driving it. Step 12 then names the **strange loop** explicitly: the Judge and Smoother are themselves practices, and their enactments can be judged and smoothed in turn.

Step 11: The autonomic harness

Steps 9 and 10 give the Judge and Smoother everything they need *as practices* — primitives in the materials, heuristics in the bundles' understanding, friction stored in the trail. What is still missing is the *driver*: something that watches for work, enacts each LLM turn, and exits when the inboxes are empty. Step 11 adds that driver. It has three pieces: an **inbox + dispatcher** pattern, a small **adapter ABC** with concrete subclasses spanning two providers and two process shapes, and one **shared run-loop** that uses them all.

Inbox tables and the dispatcher

The trail already records enactments and Friction. Step 11 adds two **inbox** tables alongside them — **judge_inbox** and **smoother_inbox**. An inbox row is a unit of pending work; it points at the underlying enactment or Friction, carries claim/lease columns so an interrupted worker doesn't strand work, and gets marked **consumed_at** when an autonomic enactment finishes processing it.

The **dispatcher** is an **asyncio** task that runs inside the server. Every couple of seconds it executes two idempotent SQL statements:

```
INSERT OR IGNORE INTO judge_inbox(enactment_id, bundle_id, closed_at,
routed_at)
SELECT id, practice_id, closed_at, NOW() FROM enactments
WHERE closed_at IS NOT NULL;

INSERT OR IGNORE INTO smoother_inbox(friction_id, target_enactment_id,
kind, emitted_at, routed_at)
SELECT id, target_enactment_id, kind, observed_at, NOW() FROM
friction_observations;
```

Both rely on **INSERT OR IGNORE** against the inbox's primary key, so re-routing the same source row is a no-op. There is no "have I already routed this" cursor to maintain — the schema is the cursor.

Workers connecting to the autonomic MCP endpoint then claim from the inboxes: **SELECT ... WHERE consumed_at IS NULL AND (claim_expires_at IS NULL OR claim_expires_at < NOW()) ORDER BY routed_at LIMIT 1**, then **UPDATE ... SET claimed_at = NOW(), claimed_by = worker_id, claim_expires_at = NOW() + lease**. If the worker finishes, the same row is marked consumed; if the worker dies, the lease eventually expires and another worker can claim it.

The adapter ABC

The actual *driving* of an LLM is the part that varies most between providers. The harness abstracts it as a small ABC:

```
class AutonomicAdapter(ABC):
    async def open(self) -> None: ...
    async def dispatch(self, work: WorkItem) -> str | None: ...
    async def close(self) -> None: ...
```

open() sets up the LLM primitive once. **dispatch(work)** is called for each inbox row and is expected to make the LLM enact one work item end-to-end. **close()** tears down.

Four concrete subclasses are provided. Three drive a real LLM, spanning two provider families (Anthropic and Codex) and two process shapes (long-lived in-process, subprocess per dispatch); the fourth, **ScriptedAdapter**, is a deterministic stand-in used by the verify so the loop can run without API keys. When the rest of the essay says "three real adapters" it means the three LLM-driving ones:

ScriptedAdapter — deterministic, no LLM. Takes a Python async callable that opens its own MCP session, drives the autonomic surface, and returns the consumer enactment id. Used by the verify so the loop runs without API keys or external tooling. The handler is what an LLM enactment *would* do, written explicitly.

AnthropicSDKAdapter — Anthropic, in-process. Uses **claude-agent-sdk** to open a long-lived **ClaudeSDKClient** per role with the bundle's brief as the system prompt; the conversation and cached prompt persist across work items. Each **dispatch** sends a single query naming the inbox row and drains the response. Requires **claude-agent-sdk** installed and Claude credentials (subscription or API key). *Billing/auth note: per Anthropic's [help center](#), from **15 June 2026** Claude Agent SDK and **claude -p** usage is metered separately from normal Claude-plan limits, with distinct Agent SDK credit behaviour. The adapter does not change; this is an operator-side cost question. Confirm the current policy at the [help-centre link](#) before pointing the adapter at a paid account.*

ClaudeCliAdapter — Anthropic, subprocess. Invokes **claude -p** (Claude Code's print mode) as a subprocess per work item. Stateless across dispatches; same provider as the SDK adapter but no Python dependency. Each call spawns a fresh **claude** process with the bundle's brief as **--system-prompt** and the dispatch message as the prompt argument; MCP config is injected via **--mcp-config** as JSON. Uses whatever credentials the CLI itself has — subscription OAuth by default, or **ANTHROPIC_API_KEY** if set. Requires the **claude** binary on PATH (configurable via **PRACTICE_CLAUDE_BIN**).

CodexExecAdapter — Codex, subprocess. Invokes **codex exec** as a subprocess per work item. Stateless across dispatches. Each call spawns a fresh process with the bundle's brief plus the work's dispatch message; the autonomic MCP server is injected inline via **codex exec -c mcp_servers....** so the adapter does not depend on the user's **~/ .codex/config.toml** or a **.mcp.json** in cwd. Requires the Codex CLI binary; configurable via **PRACTICE_CODEX_BIN**.

Briefs from bundle content

The "system prompt" the LLM enacts is not stored separately — it is *composed* from the role's bundle content. **compose_brief(bundle, substrate)** projects the bundle and formats teleo-affective + understanding + rules + affordance list into a single string. Whatever the Smoother bundle's understanding says about handling **narrow_engagement** is what an LLM enacting the Smoother sees in its prompt. Edit the bundle, the prompt changes on the next run — no second source of truth.

The shared run-loop

run_role_loop(adapter, policy, store, *, stop, worker_id) is the loop. It calls **adapter.open()** once, then loops: claim next inbox row via **policy.next_work**, call **adapter.dispatch(work)**, mark consumed, repeat. It does not care whether the adapter is Scripted, Anthropic, or Codex — the structure of "claim, dispatch, consume" is identical across them. A bounded variant **drain(adapter, policy, store, ..., max_items=N)** is provided for tests and the verify.

`RolePolicy(role="judge"|"smoother")` knows how to read each inbox and how to mark each consumed, including building the `WorkItem` from the inbox row.

The verify

The verify runs:

1. **Somatic walk** (as before). Closes practice enactments via the server's shutdown hook so the dispatcher can route them.
2. **Autonomic loop** — `verify_autonomic_loop()` opens its own `EnactmentStore`, calls `route_now(store)` to drain pending source rows into inboxes (since the somatic and autonomic servers run as separate stdio processes, the autonomic process's dispatcher would otherwise need to catch up via polling), then runs `drain(ScriptedAdapter(_scripted_judge_handler), ...)` and `drain(ScriptedAdapter(_scripted_smoother_handler), ...)`.

The output of the verify shows the inbox counts going up and down: `judge_inbox +N` after routing, then `Judge drained N work item(s)`, then `smoother_inbox +M` after Judge ran (because Judge emitted Friction), then `Smoother drained M work item(s)`. The trail at the end shows the Judge and Smoother autonomic enactments alongside the somatic ones, all top-level, with their steps recorded.

Running with a real LLM

The three real adapters are present and runnable via `autonomic_runner.py`. The shape of the MCP transport follows from the shape of the adapter.

One caveat to land before any of these commands. The dispatcher as written routes every closed enactment — somatic *and* autonomic — into `judge_inbox` with the same SQL. The verify avoids runaway recursion through bounded drains and a controlled `route_now` between passes; a continuously-polling production dispatcher does not have that gate. A Judge enactment that emits zero Friction still closes, still gets routed, still becomes another Judge work item. Without a routing cadence or filter layered over the dispatcher, the run-loop will judge the Judge's judgement of the Judge indefinitely. Step 12 ("One mechanism, two cadences") names the gap and sketches the three plausible production answers (slower routing cadence for autonomic enactments, a routing filter that drops no-Friction Judge enactments, or `PRACTICE_DISABLE_DISPATCHER=1` plus an explicit scheduler). The commands below run the *underlying mechanism* against a real LLM; the production-cadence work is still ahead. Worth knowing before you point one of these at a paid API.

AnthropicSDKAdapter — long-lived in-process. The SDK keeps a long-lived `ClaudeSDKClient` per role. The MCP transport is configurable: by default (`PRACTICE_AUTONOMIC_MCP_URL` unset) each adapter instance spawns its own stdio MCP server subprocess, which sidesteps the shared module-level state today. With `PRACTICE_AUTONOMIC_MCP_URL` set, the adapter connects to a long-lived HTTP server — the intended shape once per-session state lands. Stdio default:

```
PRACTICE_AUTONOMIC_PROVIDER=anthropic \
  uv run --extra anthropic python -m
  practice_theory_implementation.autonomic_runner
```

HTTP shape (one server, many workers, gated on per-session state):

```
# Terminal 1 – long-lived HTTP autonomic server
PRACTICE_TRANSPORT=http PRACTICE_HTTP_PORT=7181 \
  PRACTICE_SERVER_MODE=autonomic \
  uv run python -m practice_theory_implementation.server

# Terminal 2 – Anthropic SDK harness pointing at it
PRACTICE_AUTONOMIC_PROVIDER=anthropic \
  PRACTICE_AUTONOMIC_MCP_URL=http://127.0.0.1:7181/mcp/ \
  uv run --extra anthropic python -m
practice_theory_implementation.autonomic_runner
```

ClaudeCliAdapter — **subprocess per dispatch, stdio transport.** Each `claude -p` invocation spawns its own MCP server via `--mcp-config`. No separate long-lived server needed:

```
PRACTICE_AUTONOMIC_PROVIDER=anthropic_cli \
  uv run python -m practice_theory_implementation.autonomic_runner
```

If you prefer to point it at an existing long-lived HTTP server instead, set

`PRACTICE_AUTONOMIC_MCP_URL=http://.../mcp/`; the adapter switches its `--mcp-config` shape accordingly.

CodexExecAdapter — **subprocess per dispatch, stdio transport.** Each `codex exec` invocation spawns its own MCP server per dispatch via inline `-c mcp_servers...` configuration injected by the adapter. No separate long-lived server needed and no dependency on the user's Codex MCP config:

```
PRACTICE_AUTONOMIC_PROVIDER=codex \
  uv run python -m practice_theory_implementation.autonomic_runner
```

The runner drives both Judge and Smoother concurrently via `asyncio.gather(run_role_loop(...), run_role_loop(...))`, exits on SIGINT/SIGTERM or when `/tmp/practice-autonomic-quit` appears. The substrate has no idea which adapter is driving it; the difference is entirely at the adapter layer.

For the somatic side, the user's harness connects to the somatic server the same way: Codex via the `.mcp.json` entry `practice_server_somatic`, Claude Code via its MCP-server config pointing at an HTTP somatic server (`PRACTICE_TRANSPORT=http PRACTICE_SERVER_MODE=somatic`).

What step 11 contributes

Step 11 establishes the autonomic harness:

- The **inbox tables** in `trail.py`: `judge_inbox` and `smoother_inbox` with claim/lease columns; `EnactmentStore` gains the routing methods (`route_closed_enactments_to_judge_inbox`, `route_friction_to_smoother_inbox`), the atomic claim methods (`next_judge_work` / `next_smoother_work` with lease), the consume methods, and `pending*_count` helpers.

- The **dispatcher** in `autonomic_dispatcher.py`: an `asyncio` task polling on a configurable interval, plus `route_now(store)` as a synchronous one-shot for tests and the verify.
- The **adapter ABC and its four concrete subclasses** in `autonomic_adapters.py`: `ScriptedAdapter`, `AnthropicSDKAdapter`, `ClaudeCliAdapter`, `CodexExecAdapter`. `compose_brief(bundle, substrate)` builds each role's system prompt from the bundle's own content. `RolePolicy(role)` knows how to read each inbox; `drain(...)` and `run_role_loop(...)` are the two consumer shapes.
- The **autonomic runner** in `autonomic_runner.py`: an entry point that builds an adapter for each role and drives both concurrently via `asyncio.gather`.
- Server startup wiring: `asyncio.run(_serve_with_dispatcher())` runs the dispatcher alongside MCP; a shutdown handler closes any open practice enactment so the dispatcher can route it. The verify in `__main__.py` is refactored to use `ScriptedAdapter` + `drain` end-to-end.
- An optional `anthropic` dependency group in `pyproject.toml` for `claude-agent-sdk`. The two CLI adapters require their respective binaries on `PATH` (configurable via `PRACTICE_CLAUDE_BIN` and `PRACTICE_CODEX_BIN`).

Run the verify

```
uv run python -m practice_theory_implementation
```

The verify uses `ScriptedAdapter` end-to-end and prints, in order: the somatic walk (engagement projected, five practices enacted — Activities Management, Reflection, Calendar Stewardship, Practice Management, and the runtime-authored Quick Glance), the routed inbox counts after the somatic side closes, the Judge drain producing a Friction observation, the Smoother drain consuming it, the trail showing every enactment top-level or nested, and the Friction summary moving the observation from pending to addressed.

The verify is hermetic by default: it writes the trail and substrate to a fresh temp directory on every run so the printed walk matches the documented narrative without depending on any prior state. Set `PRACTICE_TRAIL_PATH` and `PRACTICE_SUBSTRATE_PATH` together to opt into persistent local files (the trail-as-substrate-of-trust story this essay describes lives there); setting only one is rejected at startup, since a mixed persistent/temp setup leaks stale state between the two halves.

What step 11 leaves for later

The autonomic loop now runs through one revolution — the user's somatic work generates closed enactments, the dispatcher routes them, the Judge enacts and emits Friction, the dispatcher routes the Friction, the Smoother enacts and amends. What is still left implicit is that **the Judge's enactments and the Smoother's enactments are themselves recorded in the trail**, alongside any other enactment. The dispatcher routes them too. The Judge can examine a Judge enactment; the Smoother can amend Smoother's own bundle. Step 12 names this — the strange loop the second essay described.

Step 12: The strange loop

Everything the loop touches gets recorded the same way. A somatic enactment closes — the dispatcher routes it to `judge_inbox`. A Judge enactment closes — the dispatcher routes it to `judge_inbox` too, with

the same SQL, against the same table. There is no special case. The Judge bundle is a bundle; its enactments are enactments; the inbox has no idea what kind of work it is queueing.

This is what doc 2 named: practices in their own right, judged and smoothed alongside everything else. The strange loop is not added in step 12 — step 12 just observes that the machinery already does it. The same harness that processes the Judge's findings about Activities Management will process the Judge's findings about an earlier Judge enactment, and the same Smoother that amended Activities Management can amend the Smoother bundle itself if a Friction names it.

One mechanism, two cadences

Essay 2 named the runaway-recursion fix as **two loops**: a reactive loop responding to somatic completions, and a separately-scheduled reflective loop examining autonomic history. It said, plainly, that *when an autonomic enactment completes, no notification is dispatched*; the autonomic history was to be visited only on the reflective loop's own schedule. The implementation collapses those two loops into **one idempotent inbox plus a discipline about when to drain it**. There is no separate "reflective" loop in the code — the dispatcher's **INSERT OR IGNORE** routes every closed enactment, somatic or autonomic, to **judge_inbox** with the same SQL.

What essay 2's two-loop scheme bought — keeping the autonomic loop from consuming itself — the implementation buys differently. In the verify, the discipline is **bounded drains plus a controlled route_now**: pass 1 routes and drains the somatic completions; pass 2 routes and drains what pass 1 produced. **route_now** between passes plays the role essay 2 gave the reflective scheduled cadence — the second pass is the reflection on the first.

In production with real LLM enactment, that discipline has to come from somewhere else, and the dispatcher as written does not supply it. **judge_inbox** is fed by *closed enactments*, not by Friction; so a Judge enactment that emits zero Friction still closes, still gets routed, and still becomes another Judge work item. A continuously-polling dispatcher with no other gate will judge the Judge's judgement of the Judge, indefinitely, even when each pass produces nothing. Quieting the smoother_inbox does not quiet judge_inbox.

So the production answer is a **routing cadence or policy** layered over the same mechanism: route somatic completions continuously, but route autonomic enactments only on a slower scheduled pass (which is essay 2's reflective cadence, recovered as a routing rule rather than a separate loop); or filter routing so a Judge enactment that produced no Friction is not queued at all; or set **PRACTICE_DISABLE_DISPATCHER=1** on the autonomic side and drive reflection from an explicit scheduler. The verify demonstrates that the underlying mechanism supports recursive self-examination; it does not demonstrate that a continuously-polling production dispatcher converges. Closing that gap — picking and implementing the cadence/policy — is honest unfinished work, and the right shape for it follows essay 2's two-loop instinct, recovered as a discipline at the routing layer rather than as a second loop.

Demonstrating one revolution

The verify, after running the first pass of Judge and Smoother, calls **route_now(store)** again. The Judge and Smoother enactments produced by the first pass have closed; the dispatcher's idempotent **INSERT** picks them up and adds them to **judge_inbox**. A second **drain** of the Judge processes them.

In the verify run, this is what happens (drains are bounded to the inbox-pending count snapshot at the start of each drain — see "A note on bounding" below):

```
First pass:
  pending judge_inbox : 5          # the five closed somatic enactments
  Judge drained 5 work item(s)
  pending smoother_inbox: 1        # the one narrow_engagement Friction
  Smoother drained 1 work item(s)

Second pass (strange loop — Judge examines Judge/Smoother enactments):
  pending judge_inbox : 6          # the 5 Judge enactments + 1 Smoother
  enactment from pass 1
  Judge drained 6 work item(s) on the second pass
```

Six new judge_inbox rows — the five Judge enactments from pass 1 (one per somatic practice judged: Activities Management, Reflection, Calendar Stewardship, Practice Management, and the runtime-authored Quick Glance) plus the one Smoother enactment that addressed the **narrow_engagement** Friction. Judge processed each in turn. No new Friction came back, because the verify's deterministic heuristic (**narrow_engagement** when **used** $\leq$ 1 of multiple available affordances) does not fire for the Judge's own enactments: each Judge enactment used at least two affordances (**read_enactment_steps** plus **read_bundle**). The loop ran, found nothing to name, and went still.

That stillness matters. Within a bounded reflective pass, the Judge's discipline — observe, do not invent — is what lets the pass quiet when nothing genuinely needs attention. A real Judge enactment, reading the bundle's understanding, would apply the same discipline; the verify's heuristic is a small-but-honest stand-in.

A note on bounding

Each drain in the verify is bounded by the pending count *at the start of that drain* —

max_items=store.pending_judge_inbox_count(). This bound exists because the loop is genuinely recursive: each autonomic enactment closes and becomes another candidate for judgement. Without a bound, a single **drain** call would keep finding new work created by its own subprocesses' shutdown handlers and never terminate naturally.

Two harness choices together make this clean:

1. **Subprocess servers run with **PRACTICE_DISABLE_DISPATCHER=1****. The verify spawns each adapter's server subprocess with this set, so the subprocesses don't run their own dispatcher polls and don't add to inboxes mid-drain. Only the verify's main process routes, via **route_now**, between drains. This separates "do the work" (subprocess) from "queue the work" (verify-controlled), which is also the right separation for production (the long-lived HTTP server runs the dispatcher; workers connecting via the adapter do not).
2. **max_items matches initial pending**. Each drain processes exactly the work that was queued when it started. Work that arrives mid-drain (newly closed autonomic enactments) waits for the next pass.

In production with real LLM enactment, the run-loop variant (**run_role_loop**) is used instead — it polls forever, idle-waits when empty, and stops on a signal. The dispatcher in the long-lived server keeps inboxes

populated; the run-loop drains them as they fill. As written, a continuously polling dispatcher still needs the routing cadence or policy named above; without it, autonomic enactments can keep re-entering `judge_inbox` even when they emit no Friction.

What the strange loop buys

Two things, both visible in the trail:

First, the apparatus can amend itself. If a Smoother enactment notices that the Smoother's own bundle has a rule that is misfiring — for example, if the rule `rule_smoother_address_what_friction_names` is being interpreted too narrowly and useful amendments are being skipped — a future Judge enactment can name that as Friction (`rule_neglect` against the Smoother enactment), and a future Smoother enactment can amend the rule via `pm_amend_pool_element`. The amendment lands in the substrate overlay; the next Smoother projection inherits the change. The Smoother improves itself through the same machinery it was built with.

Second, the apparatus stays honest. Every enactment, including the autonomic ones, leaves a trail. The user can examine what the Judge did, what the Smoother amended, what Friction was named and addressed. There is nowhere for the autonomic loop to hide. Trust as enacted structure applies to the loop itself, not just to what the loop is watching. This is what the first essay set out to engineer: AI trust expressed not as the system's account of itself but as structure — walkable in both directions, from a user's question down to the function call the practitioner reached for, and from the system's self-amendment back to the Friction that prompted it.

A note on substantive heuristics

The verify uses a single Judge heuristic (`narrow_engagement`) and a single Smoother response (append to the bundle's description). A production deployment with a real LLM enactment of each bundle will be much richer. The Judge's understanding describes several kinds of Friction to look for; an LLM reading that prose decides which fits each enactment it examines. The Smoother's understanding describes how to interpret each kind and what kinds of amendment fit; an LLM reading that prose decides what to do.

The structure that makes all of this work — the trail, the inboxes, the dispatcher, the adapters, the run-loop — is the same whether the Judge is the verify's scripted heuristic or a real LLM enactment via the Anthropic SDK adapter or the Codex exec adapter. That is the test step 11's abstraction had to pass, and step 12 confirms: the loop closes, and the closing is itself recordable, observable, and amendable.

Practice makes perfect — and the loop maintains the practice

The first essay named the missing layer as a practitioner's situated awareness, and reframed its delivery as a practice. The second essay built apprenticeship around it and named the autonomic loop. This essay implemented both, step by step.

What we have at the end is a substrate of five pools and a catalog of bundles; a projection that turns a bundle plus the engagement into a self-contained practice an LLM can enact; an MCP server presenting that projection through a fixed tool surface in two modes (somatic for the user, autonomic for the loop); a trail that records every step against an enactment; meta-materials that let practices author other practices at runtime; a Judge bundle and a Smoother bundle whose primitives are small and whose heuristics live as prose in the bundles' understanding; a dispatcher and an inbox pattern that routes pending work; and an

adapter abstraction with concrete subclasses for the Anthropic SDK, the Claude CLI, and the OpenAI Codex CLI, so the loop can run against either provider in either process shape without the substrate needing to know.

The bundle maintains the practice. The loop maintains the bundle. The strange loop maintains the loop.

The repository is a working starting point. The next move is not another step in this essay — it is using the apparatus, watching it, seeing where it falls short, and letting it amend itself.

Series

This is the third of four essays in the AI Trust series:

1. *AI Trust and Situated Awareness: A Practice Theory Reframe* — [DOI](#).
2. *Practice Theory — The Apprenticeship and a Strange Loop* — [DOI](#).
3. **This essay.** *Practice Theory — The Implementation* — DOI pending Zenodo deposit.
4. *Practice Theory — A Worked Example (Calendar Stewardship)* — DOI pending Zenodo deposit.

Companion software: [practice-theory-implementation](#) — DOI pending Zenodo deposit. Source at <https://github.com/HiddenDeveloper/practice-theory-implementation>.